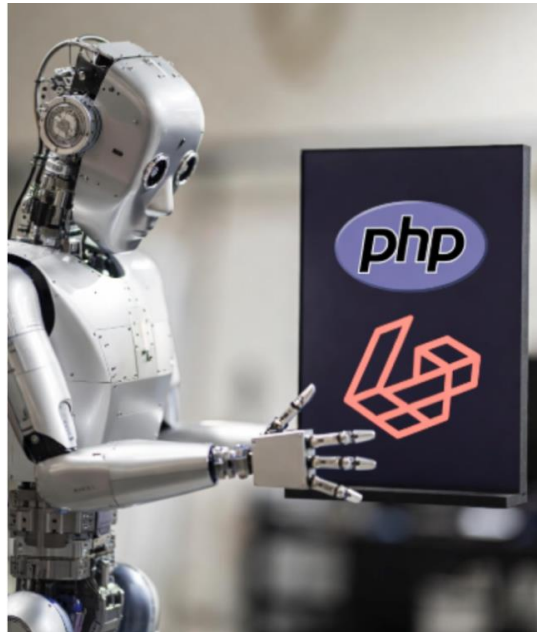




Desarrollo de Aplicaciones en Internet

LABORATORIO N° 4

Introducción a la POO



Alumno (s) :					Nota	
Grupo :			Ciclo: III			
Criterio de Evaluación	Excelente (4pts)	Bueno (3pts)	Requiere mejora (2pts)	No accept. (0pts)	Puntaje Logrado	
Entiende el uso de la POO.						
Entiende los conceptos de herencia y polimorfismos.						
Comprende la importancia de aplicar POO en una app web en PHP y documenta.						
Logra programar el desafío.						
Es puntual y redacta el informe adecuadamente.						

TEMA: INTRODUCCIÓN A PROGRAMACIÓN ORIENTADA A OBJETOS

OBJETIVOS

- Comprender los fundamentos de la Programación Orientada a Objetos (POO) y cómo aplicarlos en PHP.
- Crear clases, objetos y utilizar conceptos como encapsulamiento, herencia y polimorfismo.
- Evitar que la IA generativa les robe el pensamiento.

REQUERIMIENTOS

- **Servidor Apache instalado.**

PROCEDIMIENTO

- *El laboratorio se ha diseñado para ser desarrollado en grupos de 2.*
- *Solo un integrante sube el documento al Canvas.*
- *Entregar a tiempo para evitar la tentación del plagio.*

1. Introducción a POO.

- ¿Qué es POO? Comparación con programación procedural.
- Conceptos clave: Clases, Objetos, Propiedades, Métodos.
- Ventajas: Reutilización de código, modularidad, mantenimiento.
- Con \$this se accede a las propiedades.

```
PHP
<?php
// Definir una clase simple
class Producto {
    public $nombre = "Laptop";      // Propiedad
    public $precio = 899.99;        // Otra propiedad

    // Método
    public function mostrarInfo() {
        return "Producto: " . $this->nombre . " - Precio: $" . $this->precio;
    }
}

// Crear un objeto
$producto = new Producto();

echo $producto->mostrarInfo();
// Salida: Producto: Laptop - Precio: $899.99
?>
```

Realizar un ejemplo y explicar la funcionalidad

Funcionalidades (Lógica)
Código + Resultado

2. Encapsulamiento y Visibilidad.

- Modificadores de acceso: public, protected, private.
- Uso de getters y setters.

```
<?php

class Producto {

    // Propiedades privadas (buena práctica)
    private $nombre;
    private $precio;
    private $stock;

    // Constructor
    public function __construct($nombre, $precio, $stock = 0) {
        $this->nombre = $nombre;
        $this->precio = $precio;
        $this->stock = $stock;
    }

    // ===== GETTERS =====

    public function getNombre() {
        return $this->nombre;
    }

    public function getPrecio() {
        return $this->precio;
    }

    public function getStock() {
        return $this->stock;
    }
}
```

```
// ===== SETTERS =====

public function setNombre($nombre) {
    $this->nombre = $nombre;
}

public function setPrecio($precio) {
    if ($precio > 0) {
        $this->precio = $precio;
    } else {
        echo "Error: El precio debe ser mayor que 0.<br>";
    }
}

public function setStock($stock) {
    if ($stock >= 0) {
        $this->stock = $stock;
    } else {
        echo "Error: El stock no puede ser negativo.<br>";
    }
}
```

```
// ===== EJEMPLO DE USO =====

$producto = new Producto("Laptop HP", 1299.99, 15);

echo $producto->mostrarInfo() . "<br><br>";

$producto->setPrecio(1199.99);
$producto->setStock(10);

echo $producto->mostrarInfo() . "<br><br>";

echo $producto->vender(3) . "<br>";
echo $producto->mostrarInfo();

?>
```

Realizar un ejemplo y explicar la funcionalidad

Funcionalidades (Lógica)
Código + Resultado

3. Herencia

- ¿Qué es la herencia? Uso de extends.
- Sobrescritura de métodos.

Realizar un ejemplo y explicar la funcionalidad

Funcionalidades (Lógica)
Código + Resultado

4. Polimorfismo e Interfaces

- Interfaces: Contratos que las clases deben cumplir.
- Polimorfismo: Diferentes clases con el mismo método.

Realizar un ejemplo y explicar la funcionalidad

```
<?php

// ===== INTERFACE PADRE =====
interface Item {

    // Métodos que todas las clases que implementen deben tener
    public function getNombre();
    public function getPrecio();
    public function mostrarInfo();

    // Métodos opcionales que puedes requerir
    public function setNombre($nombre);
    public function setPrecio($precio);
}

// ===== CLASE PRODUCTO QUE IMPLEMENTA LA INTERFACE
class Producto implements Item {

    private $nombre;
    private $precio;
    private $stock;

    // Constructor
    public function __construct($nombre, $precio, $stock = 0) {
        $this->nombre = $nombre;
        $this->precio = $precio;
        $this->stock = $stock;
    }
}
```

Funcionalidades (Lógica)

Código + Resultado

5. Cree la siguiente mini app en PHP.

Descripción: Crear un sistema básico de gestión de notas. Usar clases, herencia y encapsulamiento, usar formularios.

6. ¿Qué proyecto final piensa desarrollar y cómo va usar la POO (Laravel)?

Conclusiones:

Indicar las conclusiones que llegó después de los temas tratados de manera práctica en este laboratorio.